CS260 - Written Assignment 4

Srinarayan Srikanth

Summer 2024-25

- **For any algorithm design questions in this assignment and the course as a whole, give your algorithms in C/Python-like pseudocode *and* describe your algorithm briefly at a high level in plain English.**
- Your pseudocode can omit low-level details like memory allocations and type declarations.
- If you're not sure whether or not to include a detail in your pseudocode, err on the side of including it.
- Small mistakes like off-by-one errors will receive at most a small penalty. We know it's hard to write perfectly correct pseudocode because you can't execute it easily (although running your pseudocode on an example on paper is a great way to debug it).
- Whenever we ask you to design an algorithm, we expect your algorithm to be efficient (within reason). If an asymptotically more efficient algorithm exists, you may lose points if your algorithm is correct but asymptotically slower.
- However, you should prefer a correct but slow algorithm over an incorrect attempt at a more efficient algorithm.

1. (15 points) Solve the following recurrence using the substitution/equation expansion method, then give a tight bound on the recurrence.

$$T(n) = \begin{cases} c & n \leq 0 \\ T(n-3) + 5n & n > 0 \end{cases}$$

For this problem, you may assume that sums very similar to the arithmetic series are also $O(n^2)$. The arithmetic series states that $1+2+3+4+5+...+n = O(n^2)$. Similar sums like $4+8+12+16+20+...+n$ or $7*1+7*2+7*3+7*4+7*5+...+7*n$ also sum to $O(n^2)$.

Hint: If you're getting lost in the weeds of the algebra, zoom out and think at a high level. How many substitutions/recursions will it take to reach the base case (logarithmic, linear, quadratic, etc.)? And how much work is done each substitution/recursion? You still need to solve the recurrence using the substitution method, but understanding what the final bound should be will help guide you.

Solution:

We expand by repeated substitution. Let k denote the number of substitutions performed.

$$
\begin{aligned}
T(n) &= T(n-3) + 5n \\
&= \big[T(n-6) + 5(n-3)\big] + 5n \\
&= T(n-6) + 5(n-3) + 5n \\
&= \big[T(n-9) + 5(n-6)\big] + 5(n-3) + 5n \\
&= T(n-9) + 5(n-6) + 5(n-3) + 5n \\
&\vdots \\
&= T(n-3k) + 5\sum_{i=0}^{k-1}(n-3i)
\end{aligned}
$$

We reach the base case when $n - 3k \leq 0$, i.e. when $k = \lceil n/3 \rceil \approx n/3$.

Substituting $k = n/3$:

$$
\begin{aligned}
T(n) &= c + 5\sum_{i=0}^{n/3-1}(n-3i) \\
&= c + 5\left[\frac{n}{3}\cdot n \;-\; 3\cdot\frac{(n/3)(n/3-1)}{2}\right] \\
&= c + 5\left[\frac{n^2}{3} - \frac{n(n-3)}{6}\right] \\
&= c + 5\cdot\frac{2n^2 - n^2 + 3n}{6} \\
&= c + 5\cdot\frac{n^2+3n}{6}
\end{aligned}
$$

The sum $5n + 5(n-3) + 5(n-6) + \cdots$ is an arithmetic series with $\Theta(n)$ terms, each of size $O(n)$, so it is $\Theta(n^2)$ by the given series hint.

$$\boxed{T(n) = \Theta(n^2)}$$

2. Consider a directed graph $G = (V, E)$ and its set of strongly connected components S. For each of the following questions, give the answer and explain a specific situation in which the answer would occur.

   For example, if the question asked, "What's the maximum height of a BST with n nodes?", you could say "$n - 1$, which will occur when the elements are inserted in sorted or reverse sorted order".

   (a) (8 points) **Minimum** possible size of S if there are V nodes

   Solution:

   The minimum size of S is **1**.

   This occurs when the entire graph forms a single strongly connected component, meaning every node is reachable from every other node. A concrete example: a directed cycle on V nodes, $1 \to 2 \to 3 \to \cdots \to V \to 1$. Every node can reach every other node by following the cycle, so there is exactly one SCC containing all V nodes.

   (b) (8 points) **Maximum** possible size of S if there are V nodes

   Solution:

   The maximum size of S is V.

This occurs when every node is its own SCC, meaning no two distinct nodes can reach each other. A concrete example: a directed path $1 \to 2 \to 3 \to \cdots \to V$ with no back edges. Node i can reach node j for $j > i$, but node j cannot reach node i, so no two nodes are mutually reachable. Each node forms its own SCC, giving $|S| = V$.

3. Consider a directed graph $G = (V, E)$ that has k strongly connected components. Say we add one edge to G.

   (a) (7 points) Is it possible that after adding one edge to G, k won't change?

   If yes, give a specific example of a graph and an edge where adding that edge wouldn't change k. If no, explain why.

   Solution:

   Yes, it is possible.

   Adding an edge within an existing SCC, or adding an edge from one SCC to another without creating a new cycle, leaves all SCC boundaries unchanged.

   Concrete example: Let G have 3 nodes $\{1, 2, 3\}$ with edges $1 \to 2$ and $2 \to 3$. Each node is its own SCC, so $k = 3$. Now add the edge $1 \to 3$. There is still no path from 3 back to 1 or 2, and no path from 2 back to 1. No new cycles are created, so each node remains its own SCC and $k = 3$ is unchanged.

   (b) (7 points) Is it possible that after adding one edge to G, k decreases by $V - 1$?

   If yes, give a specific example of a graph and an edge where adding that edge would decrease k by $V - 1$. If no, explain why.

   Solution:

   Yes, it is possible.

   Concrete example: Let G have V nodes arranged in a directed path: $1 \to 2 \to 3 \to \cdots \to V$. Each node is its own SCC, so $k = V$. Now add the single edge $V \to 1$. This closes the path into a directed cycle $1 \to 2 \to \cdots \to V \to 1$, making every node reachable from every other node. All V nodes collapse into one SCC, so the new $k = 1$. The decrease is $V - 1$.

   The key insight is that adding a single "closing" edge can complete a Hamiltonian cycle in the condensation, merging all V SCCs simultaneously.

4. (20 points) You're given an undirected, weighted graph G and a minimum spanning tree (MST) T of G. We then decrease the weight of some edge (u, v) in G, where (u, v) is **not** in T.

   Design an algorithm that computes a new MST of G, given the decreased weight of (u, v), in $O(V)$ time. Briefly explain why your algorithm works and why it runs in $O(V)$ time.

   If your algorithm reuses other algorithms, you must reproduce their (potentially modified) pseudocode in full.

   Hint: Since your algorithm must run in $O(V)$ time, you cannot use Prim's or Kruskal's to solve this problem. Consider how you might use T, which is already an MST, to find the new MST.

   Solution:

   High-level description: Since (u, v) is not in T, the MST T already contains a unique path P between u and v. By the MST cycle property, (u, v) should be in the MST if and only if its weight is strictly less than the maximum-weight edge on path P. If so, we swap: remove the heaviest edge on P and insert (u, v). Otherwise T is still optimal.

   Pseudocode:

```
UpdateMST(T, u, v, new_weight):
    # Step 1: Find the path from u to v in T using BFS/DFS
    parent = array of size V, initialized to -1
    visited = array of size V, initialized to False
```

```
queue = [u]
visited[u] = True

while queue is not empty:
    node = queue.pop_front()
    if node == v:
        break
    for each neighbor w of node in T:
        if not visited[w]:
            visited[w] = True
            parent[w] = node
            queue.append(w)

# Step 2: Reconstruct path and find max-weight edge on path P
max_weight = -infinity
max_edge = None
cur = v
while cur != u:
    prev = parent[cur]
    edge_weight = weight of edge (prev, cur) in T
    if edge_weight > max_weight:
        max_weight = edge_weight
        max_edge = (prev, cur)
    cur = prev

# Step 3: Decide whether to update T
if new_weight < max_weight:
    remove max_edge from T
    add edge (u, v) with weight new_weight to T

return T
```

Correctness: This follows directly from the MST cycle property. Adding (u, v) to T creates exactly one cycle — the cycle formed by (u, v) and the unique path P from u to v in T. The MST cycle property states that for any cycle in a graph, the maximum-weight edge on that cycle is not in any MST. Therefore, after decreasing the weight of (u, v):

- If new_weight $\geq$ max-edge weight on P: (u, v) is still the heaviest edge on the cycle, so T remains optimal.
- If new_weight $<$ max-edge weight on P: the old max-weight edge is now the heaviest and should be removed; (u, v) is added instead.

Runtime: T is a spanning tree on V nodes with $V - 1$ edges. BFS/DFS on T to find the path from u to v visits each node and edge at most once: $O(V)$. Tracing back the path through `parent[]` pointers and finding the maximum-weight edge is also $O(V)$. Adding and removing one edge is $O(1)$. Total runtime: $O(V)$.

5. Consider a weighted, directed graph G_1 where all edge weights are positive integers.

   (a) (10 points) Say we created a new graph G_2 by **adding** 2 to all of G_1's edge weights. Would G_1 and G_2 have the same shortest paths? Briefly explain your answer.

   Solution:

   No, G_1 and G_2 do not necessarily have the same shortest paths.

Adding a constant to every edge weight penalizes paths with more edges proportionally more than paths with fewer edges, which can change which path is shortest.

Counterexample: Let G_1 have three nodes A, B, C with:

- $A \to C$: weight 3
- $A \to B$: weight 1, $B \to C$: weight 1 (total path weight: 2)

In G_1, the shortest path from A to C is $A \to B \to C$ with weight 2.

In G_2 (add 2 to all weights):

- $A \to C$: weight 5
- $A \to B$: weight 3, $B \to C$: weight 3 (total: 6)

Now the shortest path from A to C is the direct edge $A \to C$ with weight 5. The shortest paths differ between G_1 and G_2.

Intuition: Adding a constant c to every edge adds $c \cdot$ (number of edges) to a path's total weight. A path through more edges gets penalized more, breaking ties that existed in G_1.

(b) (10 points) Say we created a new graph G_2 by **multiplying** all of G_1's edge weights by 2. Would G_1 and G_2 have the same shortest paths? Briefly explain your answer.

Solution:

Yes, G_1 and G_2 have the same shortest paths.

Multiplying every edge weight by the same positive constant $c = 2$ scales the total weight of every path by exactly c. If path P_1 has weight w_1 and path P_2 has weight w_2 in G_1, then in G_2 they have weights $2w_1$ and $2w_2$. Since $2 > 0$, the inequality $w_1 < w_2$ holds if and only if $2w_1 < 2w_2$. Therefore, the relative ordering of all paths between every pair of nodes is identical in G_1 and G_2, so the shortest paths are the same.

6. (15 points) Design an $O(V(V + E))$ algorithm that solves the all pairs shortest path problem on a weighted graph where all edge weights are 2.

Briefly explain why your algorithm is correct and runs in time $O(V(V + E))$.

If your algorithm reuses other algorithms given in the class with some tweaks, you **don't** have to reproduce their pseudocode in full – just describe in detail any changes you'd make.

Solution:

High-level description: Since all edge weights are equal (weight 2), the shortest path between any two nodes is simply the path that uses the fewest edges, scaled by 2. BFS finds shortest paths in terms of edge count in $O(V + E)$. Running BFS from every source vertex solves the all-pairs problem in $O(V(V + E))$.

Algorithm:

For each vertex $s \in V$:

1. Run standard BFS from s on G (treating G as unweighted).
2. For each vertex $t \in V$, set dist$[s][t] =$ BFS_distance$(s, t) \times 2$.
3. If t is unreachable from s, set dist$[s][t] = \infty$.

No modifications to BFS are needed beyond recording the hop-distance from the source to each node.

Correctness: Since every edge has weight 2, the total weight of any path is 2×(number of edges on the path). Minimizing total weight is therefore equivalent to minimizing the number of edges, which is exactly what BFS computes. BFS guarantees that `BFS_distance`(s, t) is the minimum number of edges on any path from s to t, so multiplying by 2 gives the true shortest-path weight.

Runtime: Standard BFS on a graph with V vertices and E edges runs in $O(V + E)$. We run BFS once from each of the V source vertices, giving a total runtime of $O(V(V + E))$.